

SOLID Principles

Complete interview question guide for Junior .NET Developers

Principle	Full Name	Questions
S	Single Responsibility Principle	4
O	Open/Closed Principle	4
L	Liskov Substitution Principle	4
I	Interface Segregation Principle	4
D	Dependency Inversion Principle	5

S — Single Responsibility Principle

One class, one reason to change

Q1. What is the Single Responsibility Principle (SRP)?

Basic

Answer: A class should have only one reason to change — it should be responsible for exactly one part of the system's functionality. If a class handles multiple concerns, a change in one concern may unexpectedly break another.

Bad — violates principle

```
class UserService {  
    void SaveUser() { ... }  
    void SendEmail() { ... }  
    void GenerateReport() { ... }  
}
```

Good — follows principle

```
class UserRepository { void SaveUser() { ... } }  
class EmailService { void SendEmail() { ... } }  
class ReportGenerator { void  
    GenerateReport() { ... } }
```

Q2. How would you identify if a class is violating SRP?

Intermediate

Answer: Signs of SRP violation: the class has many methods from unrelated domains, changes to one feature require editing this class, the class has many injected dependencies, or it is very long (100+ lines often hints at multiple responsibilities).

Q3. Does SRP apply only to classes, or also to methods?

Intermediate

Answer: SRP applies at all levels — classes, methods, and modules. A method should do one thing. If a method name contains 'And' (e.g. SaveAndSendEmail), it likely violates SRP and should be split into two separate methods.

Q4. What is the difference between SRP and Separation of Concerns?

Advanced

Answer: They are related but not identical. Separation of Concerns (SoC) is a broader architectural principle — different parts of a system handle different concerns (UI, business logic, data access). SRP is a class-level rule derived from SoC. SRP is the OOP implementation of SoC.

O — Open/Closed Principle

Open for extension, closed for modification

Q5. What is the Open/Closed Principle (OCP)?

Basic

Answer: A class should be open for extension (you can add new behavior) but closed for modification (you should not change existing tested code). This is typically achieved through inheritance, interfaces, and polymorphism.

Bad — violates principle	Good — follows principle
<pre>void DrawShape(string type) { if (type == "circle") ... else if (type == "square") ... // Must edit here to add Triangle }</pre>	<pre>interface IShape { void Draw(); } class Circle : IShape { public void Draw() { ... } } class Triangle : IShape { public void Draw() { ... } } // Adding new shape = new class only</pre>

Q6. How do you apply OCP in a real ASP.NET Core project?

Intermediate

Answer: Use interfaces and dependency injection. For example, a payment system defines IPaymentGateway. Adding Stripe or PayPal support means creating a new class implementing that interface — no existing code is touched. Middleware pipelines in ASP.NET Core also follow OCP.

Q7. What design patterns help implement OCP?

Advanced

Answer: Strategy pattern: encapsulate algorithms behind an interface so new ones can be added. Decorator pattern: wrap objects to add behavior without modification. Template Method: define a skeleton in a base class, let subclasses fill in steps. Factory/Abstract Factory: create objects without specifying exact types.

Q8. Can over-applying OCP be a problem?

Advanced

Answer: Yes. Premature abstraction leads to unnecessary complexity — creating interfaces for every class 'just in case' makes code harder to read. Apply OCP when you anticipate change, or after the second time you need to modify a class for similar reasons (the Rule of Three).

L — Liskov Substitution Principle

Derived types must be substitutable for base types

Q9. What is the Liskov Substitution Principle (LSP)?

Basic

Answer: If S is a subtype of T, objects of type T may be replaced with objects of type S without altering program correctness. A derived class should behave correctly wherever its base class is expected — no surprises.

Bad — violates principle	Good — follows principle
<pre>class Bird { public virtual void Fly() { ... } } class Penguin : Bird { public override void Fly() => throw new NotSupportedException(); // LSP violation! }</pre>	<pre>interface IFlyable { void Fly(); } class Eagle : IFlyable { public void Fly() { ... } } class Penguin { /* No Fly — penguins don't fly */ }</pre>

Q10. What rules must a subclass follow to not violate LSP?

Intermediate

Answer: 1. Preconditions cannot be strengthened (derived method cannot require more). 2. Postconditions cannot be weakened (must deliver at least what base promises). 3. Invariants of the base class must be preserved. 4. No throwing new exceptions not in the base contract. 5. Return types must be covariant.

Q11. Explain the classic Rectangle-Square LSP problem.

Intermediate

Answer: Square inherits Rectangle. Code sets Width=5, Height=10 expecting Area=50, but Square overrides setters to keep sides equal, so Area=100 — unexpected. Square is NOT substitutable for Rectangle. Solution: use separate classes or an IShape interface instead of inheritance.

Q12. How do you detect LSP violations in code reviews?

Advanced

Answer: Look for: overridden methods that throw NotSupportedException or NotImplementedException, methods that do nothing in derived classes, code that type-checks with 'is' or 'as' before calling a method, and unit tests that pass for the base class but fail for the derived class.

I — Interface Segregation Principle

Clients should not depend on interfaces they don't use

Q13. What is the Interface Segregation Principle (ISP)?

Basic

Answer: Clients should not be forced to implement interfaces they don't use. Instead of one large 'fat' interface, create several smaller focused ones. This prevents classes from having empty stubs or NotImplementedException methods.

Bad — violates principle

```
interface IWorker {
    void Work();
    void Eat();
    void Sleep();
}
class Robot : IWorker {
    public void Eat() => throw new
    NotSupportedException();
}
```

Good — follows principle

```
interface IWorkable { void Work(); }
interface IEatable { void Eat(); }
class Human : IWorkable, IEatable { ... }
class Robot : IWorkable { ... }
```

Q14. How is ISP different from SRP?

Intermediate

Answer: SRP is about classes having one responsibility. ISP is about interfaces being focused so implementing classes are not burdened with unused methods. A class can follow SRP but still implement a fat interface that violates ISP. They complement each other.

Q15. Give a real example of ISP in the .NET BCL or ASP.NET Core.

Intermediate

Answer: IEnumerable vs ICollection vs IList is a classic example. IEnumerable only exposes iteration. ICollection adds Count, Add, Remove. IList adds index access. Each client takes only what it needs. In ASP.NET Core, IHostedService (Start/Stop only) vs more specific interfaces follows the same idea.

Q16. Can applying ISP lead to too many interfaces? How do you balance it?

Advanced

Answer: Yes — interface explosion is a real risk. Balance by grouping methods that are always used together. Use the 'role interface' pattern: define interfaces based on what roles a client needs. If two

methods are always called together, they belong in one interface.

D — Dependency Inversion Principle

Depend on abstractions, not concretions

Q17. What is the Dependency Inversion Principle (DIP)?

Basic

Answer: 1. High-level modules should not depend on low-level modules — both should depend on abstractions. 2. Abstractions should not depend on details; details should depend on abstractions. In practice: depend on interfaces, not concrete classes.

Bad — violates principle

```
class OrderService {  
    // Tight coupling to concrete class  
    private SqlOrderRepository _repo  
    = new SqlOrderRepository();  
}
```

Good — follows principle

```
class OrderService {  
    private readonly IOrderRepository _repo;  
    public OrderService(IOrderRepository repo) {  
        _repo = repo; // injected abstraction  
    }  
}
```

Q18. What is the difference between DIP, Dependency Injection, and IoC?

Intermediate

Answer: DIP is a design principle — depend on abstractions. Inversion of Control (IoC) is a broader pattern — the framework calls your code, not the other way around. Dependency Injection (DI) is one technique to achieve IoC — dependencies are provided rather than created internally. DIP is the 'why', DI is the 'how'.

Q19. How does ASP.NET Core's built-in DI container relate to DIP?

Intermediate

Answer: ASP.NET Core's *IServiceCollection*/*IServiceProvider* implements IoC via constructor injection. You register *IOrderRepository* -> *SqlOrderRepository* in *Program.cs*, and the framework injects the concrete class wherever *IOrderRepository* is requested — perfectly implementing DIP without tight coupling.

Q20. What are the DI lifetime options in ASP.NET Core and when do you use each?

Intermediate

Answer: Transient: new instance every request. Best for lightweight, stateless services. Scoped: one instance per HTTP request. Best for *DbContext*. Singleton: one instance for app lifetime. Best for caches and thread-safe services. Injecting a Scoped service into a Singleton causes a 'captive dependency' bug.

Q21. What is a 'captive dependency' and why is it dangerous?

Advanced

Answer: A captive dependency occurs when a longer-lived service (Singleton) holds a reference to a shorter-lived one (Scoped). The scoped service gets captured and lives longer than intended, causing stale data or concurrency bugs. ASP.NET Core detects this at runtime in development mode and throws *InvalidOperationException*.

Interview tip: For each principle, state the definition, give a real-world scenario, then show violation vs correct code.